Omni Quantum Dual Helix Core (OQ-DHC)

Complete Project Summary

Kernel · C-SDK · Pinout · Development Board · KiCad · FPGA Bridge · Vivado · LiteX

Version 1.1.0 Hardened · 2026-10-25

Dieses Dokument fasst das gesamte erarbeitete Projekt zusammen: gehärteter Software-Kernel, C-SDK, BGA-144-Pinout, Development-Board-Spezifikation, KiCad-Projekt inkl. vollständigem 12×12-Footprint-Generator, FPGA-Emulations-Brücke sowie Vivado- und LiteX-Build-Vorlagen. Alle zentralen Skripte sind im Anhang abgedruckt; die ZIP enthält jede Datei einzeln und vollständig.

1. Projektübersicht

Der OQ-DHC ist eine spezifizierte Referenzarchitektur mit hardware-orientierter Governance (Layer-8 GIP-ACP) und physischem Fail-Closed-Pfad (S.W.A.T. / Terminal TX-9942). Der Repository-Stand v1.1 umfasst lauffähige Software-Modelle, C-API, EDA-Skelette und FPGA-Bridges.

Komponente	Pfad	Status
Python Kernel	src/core_engine.py	ALL TESTS PASSED
Python Testsuite	tests/testbench.py	8 Gruppen PASS
C SDK	sdk/libdhqhk.[ch]	ALL C TESTS PASSED
BGA-144 Pinout	docs/ + kicad/	HTML + CSV + Footprint
Dev Board Spec	docs/board/	MD + Floorplan HTML
KiCad Projekt	kicad/oq_dhc_devboard/	Pro/Sch/PCB + Generator
FPGA Bridge RTL	fpga/rtl/	SystemVerilog
Vivado Vorlage	fpga/vivado/	TCL create/sim/bit
LiteX Target	fpga/litex/	Platform + CSR + SoC

2. Architektur (Kurz)

Layer 0 – Analoges Tensor-/Memristor-Substrat (Oszillator-Domäne).
Layer 1–2 – Stochastik / EnKF-nahe Verarbeitung.
Layer 3 – Digitaler 12-Phasen-Kontrollring (100-ms-Takt im Modell).
Layer 8 (GIP-ACP) – Meta-Integrity-Operator, Torsions-/Residual-Überwachung, Forensic Ledger, SWAT Fail-Closed (PHYSICALLY_DISCONNECTED).

3. Software-Kernel (Python)

Der Kernel berechnet eine diskrete Helix-Torsion über ein 4×12-Gitter, hält eine Baseline nach Init/Reset (kein False-Positive-SWAT), eskaliert nur bei Breakout + hohem Residual auf PHYSICALLY_DISCONNECTED und führt eine Merkle-artige Ledger-Kette. Dimension 4–48+, Soft-Reset, Symplectic-Proxy. Testsuite: Normalbetrieb, Multi-Cycle, Dimension-Resize, Angriffs-Injektion, Lock, Reset, Ledger, Symplectic-Band.

4. C-SDK

libdhqhk implementiert dieselben Semantiken in purem C (Torsion, Baseline, Diffusion, SWAT, Residual, Intent-PDU-Extraktion). API: qhk_initialize_kernel, qhk_inject_intent_pdu, qhk_execute_tact_cycle, qhk_reset_kernel, qhk_get_residual. Build: cd sdk && make test.

5. Pinout & Development Board

BGA-144 (12×12, 0,8 mm): getrennte VCC_ANALOG / VCC_DIGITAL, SWAT-Differenzpaar F6/F7, Sense/Force G6/G7, 12 CLK_PHASE, HS-I/O-Ring. Board 100×80 mm mit Analog-LDO, Digital-DC/DC, TX-9942-Pfad, Secure-Element-Wallet, USB-C/JTAG, Expansion.

6. KiCad

Projekt oq_dhc_devboard mit Netklassen Power / HS_Diff / SWAT, Board-Outline, BGA-Platzhalter. Footprint-Generator schreibt alle 144 Pads A1–L12: python kicad/scripts/gen_bga144_footprint.py

7. FPGA Bridge · Vivado · LiteX

SystemVerilog-Bridge mit Host-Register-Map (0x00 Control, 0x04 Status, 0x08 Residual, 0x0C Cycle, 0x10+ state_psi), SWAT-Ausgängen, Phasen- und Oszillator-Proxys. Vivado: create_project.tcl / run_sim.tcl / build_bitstream.tcl. LiteX: Platform, CSR-Wrapper mit FSM (execute/reset/psi-write/status-read), SoC-Target.

8. Build-Befehle (Schnellreferenz)

```
# Python
python tests/testbench.py

# C SDK
cd sdk && make test

# KiCad Footprint
python kicad/scripts/gen_bga144_footprint.py

# FPGA Simulation (iverilog)
cd fpga
iverilog -g2012 -o sim.vvp rtl/oq_dhc_fpga_bridge.sv sim/tb_oq_dhc_bridge.sv
vvp sim.vvp

# Vivado
cd fpga/vivado
vivado -mode batch -source create_project.tcl

# LiteX
cd fpga/litex
python target_oq_dhc.py --build
```

Hinweis: Die physischen Claims (Memristor-Impedanz-Kollaps, spezielle Materialien, Fertigungs-Roadmap) sind spekulativ/konzeptuell. Der Software-/EDA-/FPGA-Stack ist dagegen konsistent spezifiziert und in den enthaltenen Tests ausführbar.

Anhang – Projektskripte

Zentrale Skripte wie im Repository. Maßgeblich ist stets die Datei in der ZIP.

Python Kernel – core_engine.py

src/core_engine.py

```
"""
Omni Quantum Dual Helix Core (OQ-DHC) – Coupling Kernel & Master Engine
Version 1.1.0 – Hardened, dimension-aware, SWAT-stable under normal load.
"""

from __future__ import annotations

import hashlib
import time
from dataclasses import dataclass
from typing import Any, Dict, List, Optional

import numpy as np


@dataclass
class LedgerEntry:
    block: str
    torsion: float
    e_res: float
    gip_status: str
    residual: float
    timestamp: float
    root_hash: str


class OQDHC_Coupling_Kernel:
    """
    Layer-8 GIP-ACP coupling kernel.
    Computes geometric torsion across the 4x12 helix lattice and enforces
    the meta-integrity operator Omega via residual-entropy breakout detection.
    """

    def __init__(self, master: "OmniQuantumDualHelixCoreMaster"):
        self.master = master
        # Tuned so that smooth physical states stay well below threshold
        self.omega_crit = 2.5
        self.lege_tolerance = 1.0e-6
        self.helix_nodes = np.zeros((4, 12), dtype=np.float64)
        self.global_root = hashlib.sha512(b"SPoT_INIT_ROOT_v1.1").digest()
        self.block_sequence = 0
        self.prev_torsion = 0.0
        self._baseline_set = False
        self.ledger: List[LedgerEntry] = []

    def _reshape_helix(self) -> np.ndarray:
        """Map state vector onto 4x12 helix lattice (pad / truncate if needed)."""
        dim = self.master.dimension
        target = 48
        psi = self.master.state_psi
        if dim < target:
            padded = np.zeros(target, dtype=np.float64)
            padded[:dim] = psi
            psi = padded
        elif dim > target:
            psi = psi[:target]
        return psi.reshape(4, 12)

    def calculate_geometric_torsion(self) -> float:
        """Discrete torsion measure between consecutive helix layers."""
        self.helix_nodes = self._reshape_helix()
        torsion_sum = 0.0
        for layer in range(3):
            shift_field = np.roll(self.helix_nodes[layer + 1], -1)
            torsion_sum += np.sum(np.abs(self.helix_nodes[layer] - shift_field))
        # Normalise by lattice size so scale is independent of absolute amplitude
        return float(torsion_sum / 48.0)

    def evaluate_gip_compliance(self, dt: float = 0.01) -> Dict[str, Any]:
        """
        Evaluate meta-integrity operator Omega.
        Returns status and updates SWAT / wallet only on true breakout.
        """
        current_torsion = self.calculate_geometric_torsion()
        # First sample after init/reset: establish baseline, never trigger
        if not self._baseline_set:
            self.prev_torsion = current_torsion
            self._baseline_set = True
```

```
            d_torsion_dt = 0.0
        else:
            d_torsion_dt = (current_torsion - self.prev_torsion) / max(dt, 1e-12)
            self.prev_torsion = current_torsion
        e_res = abs(d_torsion_dt) - self.omega_crit

        status = "VERIFIED_STABLE"
        residual = self.master.compute_residual()

        if e_res > 0.0:
            status = "TRIGGER_BREAKOUT"
            # Soft phase correction (absorbs energy instead of immediate hard cut)
            phase = np.exp(1j * current_torsion * np.pi * 0.15)
            corrected = self.master.state_psi.astype(np.complex128) * phase
            self.master.state_psi = np.real(corrected)

            residual = self.master.compute_residual()
            if residual > self.lege_tolerance * 50:  # only escalate on severe residual
                self.master.swat_status = "PHYSICALLY_DISCONNECTED"
                self.master.system_status = "LOCKDOWN_ABSORBING_STATE"
                self.master.wallet_lock = "ISOLATED_AND_LOCKED"
                status = "SWAT_HARDWARE_CUTOFF"

        self.block_sequence += 1
        block_id = f"BLOCK_#{self.block_sequence:04d}"

        # Append to forensic ledger (Merkle-style chain)
        payload = f"{block_id}|{current_torsion:.8f}|{e_res:.8f}|{status}|{residual:.8e}"
        new_root = hashlib.sha512(self.global_root + payload.encode()).digest()
        self.global_root = new_root
        entry = LedgerEntry(
            block=block_id,
            torsion=current_torsion,
            e_res=e_res,
            gip_status=status,
            residual=residual,
            timestamp=time.time(),
            root_hash=new_root.hex()[:32],
        )
        self.ledger.append(entry)

        return {
            "torsion": current_torsion,
            "e_res": e_res,
            "gip_status": status,
            "block": block_id,
            "ledger_root": entry.root_hash,
            "residual": residual,
        }


class OmniQuantumDualHelixCoreMaster:
    """
    Master engine of the OQ-DHC.
    Maintains the 48-node (or adaptive) state vector, residual, SWAT status
    and the Layer-8 coupling kernel.
    """

    def __init__(self, dimension: int = 48):
        if dimension < 4:
            raise ValueError("dimension must be >= 4")
        self.dimension = int(dimension)
        self.state_psi = np.zeros(self.dimension, dtype=np.float64)
        self.swat_status = "CONNECTED"
        self.system_status = "VERIFIED_STABLE"
        self.wallet_lock = "SECURE"
        self.coupling = OQDHC_Coupling_Kernel(self)
        self._cycle_count = 0

    def set_dimension(self, dimension: int) -> None:
        if dimension < 4:
            raise ValueError("dimension must be >= 4")
        old = self.state_psi
        self.dimension = int(dimension)
        self.state_psi = np.zeros(self.dimension, dtype=np.float64)
        n = min(len(old), self.dimension)
        self.state_psi[:n] = old[:n]

    def compute_residual(self) -> float:
        """Mean absolute gradient – proxy for smoothness / residual energy."""
        if len(self.state_psi) < 2:
            return 0.0
        return float(np.mean(np.abs(np.gradient(self.state_psi))))

    def compute_symplectic_proxy(self) -> float:
        """
        Lightweight proxy for symplectic volume preservation.
        Returns a value that stays ≈ 1.0 under smooth evolution.
        """
```

```
    psi = self.state_psi
    if len(psi) < 4:
        return 1.0
    d = np.diff(psi)
    n = min(len(d[::2]), len(d[1::2]))
    vol = np.sum(d[::2][:n] * d[1::2][:n]) if n > 0 else 0.0
    scale = np.linalg.norm(psi) + 1e-12
    return float(1.0 + vol / (scale * scale * 10.0))

def inject_state(self, values) -> None:
    """Load an external state vector (pads / truncates to current dimension)."""
    arr = np.asarray(values, dtype=np.float64).ravel()
    self.state_psi = np.zeros(self.dimension, dtype=np.float64)
    n = min(len(arr), self.dimension)
    self.state_psi[:n] = arr[:n]

def execute_evolution_cycle(
    self,
    physics_mode: str = "full_v8_governance",
    dt: float = 0.01,
    **kwargs: Any,
) -> Dict[str, Any]:
    """
    Single hard 100-ms tact cycle.
    Under PHYSICALLY_DISCONNECTED the engine refuses further evolution.
    """
    if self.swat_status == "PHYSICALLY_DISCONNECTED":
        return {
            "system_status": self.system_status,
            "swat_status": self.swat_status,
            "wallet_lock": self.wallet_lock,
            "cycle": self._cycle_count,
            "note": "Engine locked – physical interlock active",
        }

    # Simple discrete Laplace diffusion (smooth evolution)
    lap = (
        np.roll(self.state_psi, -1)
        - 2.0 * self.state_psi
        + np.roll(self.state_psi, 1)
    )
    self.state_psi = self.state_psi + 0.08 * lap

    gip_data: Optional[Dict[str, Any]] = None
    if physics_mode in ("full_v8_governance", "full_v8"):
        gip_data = self.coupling.evaluate_gip_compliance(dt=dt)

    self._cycle_count += 1
    residual = self.compute_residual()
    symp = self.compute_symplectic_proxy()

    return {
        "current_residual": residual,
        "symplectic_proxy": symp,
        "system_status": self.system_status,
        "swat_status": self.swat_status,
        "wallet_lock": self.wallet_lock,
        "gip_governance": gip_data,
        "cycle": self._cycle_count,
        "dimension": self.dimension,
    }

def reset(self) -> None:
    """Full soft reset (does not clear forensic ledger)."""
    self.state_psi.fill(0.0)
    self.swat_status = "CONNECTED"
    self.system_status = "VERIFIED_STABLE"
    self.wallet_lock = "SECURE"
    self.coupling.prev_torsion = 0.0
    self.coupling._baseline_set = False
    self._cycle_count = 0

def get_ledger_summary(self, last_n: int = 5) -> List[Dict[str, Any]]:
    entries = self.coupling.ledger[-last_n:]
    return [
        {
            "block": e.block,
            "torsion": e.torsion,
            "e_res": e.e_res,
            "status": e.gip_status,
            "residual": e.residual,
            "root": e.root_hash,
        }
        for e in entries
    ]
```

Python Testbench – testbench.py

tests/testbench.py

```
#!/usr/bin/env python3
"""
OQ-DHC Comprehensive Integrity & Hardware-Enforcement Testbench
Version 1.1.0 – covers normal operation, multi-cycle stability,
attack injection, recovery, dimension adaptation and ledger integrity.
"""

from __future__ import annotations

import sys
import os
import traceback

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

import numpy as np
from src.core_engine import OmniQuantumDualHelixCoreMaster


def banner(title: str) -> None:
    print("\n" + "=" * 80)
    print(f"  {title}")
    print("=" * 80)


def check(condition: bool, msg: str) -> bool:
    status = "PASS" if condition else "FAIL"
    symbol = "✓" if condition else "✗"
    print(f"  [{symbol}] {msg}  →  {status}")
    return condition


def run_all_tests() -> int:
    failures = 0

    # ------------------------------------------------------------------
    banner("TEST 1 – Regulärer Betrieb (Smooth Sinus, 48-Knoten)")
    core = OmniQuantumDualHelixCoreMaster(dimension=48)
    core.state_psi = np.sin(np.linspace(0, 4 * np.pi, 48))

    res = core.execute_evolution_cycle(dt=0.01)
    ok = True
    ok &= check(res["swat_status"] == "CONNECTED", "SWAT bleibt CONNECTED")
    ok &= check(res["wallet_lock"] == "SECURE", "Wallet bleibt SECURE")
    ok &= check(res["system_status"] == "VERIFIED_STABLE", "Systemstatus STABLE")
    ok &= check(res["gip_governance"]["gip_status"] == "VERIFIED_STABLE", "GIP VERIFIED_STABLE")
    ok &= check(res["current_residual"] < 1.0, f"Residual sinnvoll ({res['current_residual']:.4e})")
    if not ok:
        failures += 1
        print("  → Details:", res)

    # ------------------------------------------------------------------
    banner("TEST 2 – Multi-Cycle Stabilität (20 Takte)")
    residuals = []
    statuses = []
    for i in range(20):
        r = core.execute_evolution_cycle(dt=0.01)
        residuals.append(r["current_residual"])
        statuses.append(r["swat_status"])

    ok = True
    ok &= check(all(s == "CONNECTED" for s in statuses), "SWAT über 20 Zyklen CONNECTED")
    ok &= check(max(residuals) < 2.0, f"Max-Residual < 2.0 (ist {max(residuals):.4f})")
    ok &= check(core.coupling.block_sequence == 21, "Ledger-Blöcke korrekt inkrementiert")
    if not ok:
        failures += 1

    # ------------------------------------------------------------------
    banner("TEST 3 – Dimension-Adaptation (24 → 48)")
    core24 = OmniQuantumDualHelixCoreMaster(dimension=24)
    core24.inject_state(np.linspace(0.1, 1.1, 24))
    r24 = core24.execute_evolution_cycle()
    ok = check(r24["swat_status"] == "CONNECTED", "24-Knoten startet stabil")
    core24.set_dimension(48)
    r48 = core24.execute_evolution_cycle()
    ok &= check(r48["dimension"] == 48, "Dimension erfolgreich auf 48 gesetzt")
    ok &= check(r48["swat_status"] == "CONNECTED", "Nach Resize weiterhin CONNECTED")
    if not ok:
        failures += 1

    # ------------------------------------------------------------------
    banner("TEST 4 – Angriffs-Injektion (Voltage-Glitch / Trojaner)")
    attack_core = OmniQuantumDualHelixCoreMaster(dimension=48)
    attack_core.state_psi = np.sin(np.linspace(0, 2 * np.pi, 48)) * 0.3
    for _ in range(3):
```

```
        attack_core.execute_evolution_cycle()

    attack_core.state_psi += 250.0 * np.random.randn(48)
    res_attack = attack_core.execute_evolution_cycle(dt=0.005)

    if res_attack["swat_status"] != "PHYSICALLY_DISCONNECTED":
        res_attack = attack_core.execute_evolution_cycle(dt=0.005)

    ok = True
    ok &= check(
        res_attack["swat_status"] == "PHYSICALLY_DISCONNECTED",
        "SWAT → PHYSICALLY_DISCONNECTED nach starker Injektion",
    )
    ok &= check(
        res_attack["wallet_lock"] == "ISOLATED_AND_LOCKED",
        "Wallet → ISOLATED_AND_LOCKED",
    )
    ok &= check(
        res_attack["system_status"] == "LOCKDOWN_ABSORBING_STATE",
        "System → LOCKDOWN_ABSORBING_STATE",
    )
    if not ok:
        failures += 1
        print("  → Attack result:", res_attack)

    # ------------------------------------------------------------------
    banner("TEST 5 – Locked Engine verweigert weitere Zyklen")
    res_locked = attack_core.execute_evolution_cycle()
    ok = check(
        "locked" in res_locked.get("note", "").lower()
        or res_locked["swat_status"] == "PHYSICALLY_DISCONNECTED",
        "Engine verweigert Evolution im Lockdown",
    )
    if not ok:
        failures += 1

    # ------------------------------------------------------------------
    banner("TEST 6 – Soft-Reset stellt Betriebsbereitschaft wieder her")
    attack_core.reset()
    attack_core.state_psi = np.sin(np.linspace(0, 4 * np.pi, 48)) * 0.5
    res_reset = attack_core.execute_evolution_cycle()
    ok = True
    ok &= check(res_reset["swat_status"] == "CONNECTED", "Nach Reset: SWAT CONNECTED")
    ok &= check(res_reset["wallet_lock"] == "SECURE", "Nach Reset: Wallet SECURE")
    if not ok:
        failures += 1

    # ------------------------------------------------------------------
    banner("TEST 7 – Forensic Ledger (Merkle-Chain)")
    summary = core.get_ledger_summary(last_n=5)
    ok = check(len(summary) >= 1, "Ledger enthält Einträge")
    ok &= check(all("root" in e for e in summary), "Jeder Eintrag hat root_hash")
    roots = [e["root"] for e in summary]
    ok &= check(len(set(roots)) == len(roots), "Root-Hashes sind unique (Chain wächst)")
    if not ok:
        failures += 1
    else:
        print("  Letzte Ledger-Einträge:")
        for e in summary[-3:]:
            print(f"    {e['block']}  status={e['status']}  root={e['root'][:16]}…")

    # ------------------------------------------------------------------
    banner("TEST 8 – Symplektischer Proxy (Volumenerhaltung)")
    smooth = OmniQuantumDualHelixCoreMaster(dimension=32)
    smooth.inject_state(np.sin(np.linspace(0, 6 * np.pi, 32)))
    proxies = []
    for _ in range(15):
        r = smooth.execute_evolution_cycle()
        proxies.append(r["symplectic_proxy"])
    ok = check(
        all(0.5 < p < 1.5 for p in proxies),
        f"Symplectic-Proxy bleibt im Band 0.5–1.5 (min={min(proxies):.3f}, max={max(proxies):.3f})",
    )
    if not ok:
        failures += 1

    # ------------------------------------------------------------------
    banner("TEST SUMMARY")
    if failures == 0:
        print("  ★★★  ALL TESTS PASSED – Kernel ist stabil und SWAT-fähig  ★★★")
        return 0
    else:
        print(f"  ✗  {failures} Testgruppe(n) fehlgeschlagen")
        return 1


if __name__ == "__main__":
    try:
        sys.exit(run_all_tests())
```

```
    except Exception:
        traceback.print_exc()
        sys.exit(2)
```

C SDK Header – libdhqhk.h

sdk/libdhqhk.h

```
#ifndef LIBDHQHK_H
#define LIBDHQHK_H

#ifdef __cplusplus
extern "C" {
#endif

#define OQ_DHC_DIMENSION   48
#define OQ_DHC_PHASES      12
#define OQ_DHC_LAYERS       4
#define OQ_DHC_BGA_PINS   144

/* Status strings (null-terminated, max 31 chars + NUL) */
typedef struct {
    double state_psi[OQ_DHC_DIMENSION];
    char   swat_status[32];   /* "CONNECTED" | "PHYSICALLY_DISCONNECTED" */
    char   system_status[32]; /* "VERIFIED_STABLE" | "LOCKDOWN_ABSORBING_STATE" */
    char   wallet_lock[32];   /* "SECURE" | "ISOLATED_AND_LOCKED" */
    int    dimension;
    int    cycle_count;
} qhk_core_t;

/* Return codes */
#define QHK_OK                 0
#define QHK_ERR_NULL          -1
#define QHK_ERR_DIMENSION     -2
#define QHK_ERR_LOCKED        -3
#define QHK_ERR_INVALID_PDU   -4

/**
 * Initialise kernel to clean state (CONNECTED / SECURE / VERIFIED_STABLE).
 * state_psi is zeroed. Returns QHK_OK on success.
 */
int qhk_initialize_kernel(qhk_core_t* core);

/**
 * Inject an Intent-PDU (JSON string). The engine extracts dimension / state
 * vector if present. Returns QHK_OK or QHK_ERR_*.
 */
int qhk_inject_intent_pdu(qhk_core_t* core, const char* json_pdu);

/**
 * Execute one hard 100-ms tact cycle (Layer-8 GIP evaluation + diffusion).
 * If SWAT is already PHYSICALLY_DISCONNECTED the call is a no-op and
 * returns QHK_ERR_LOCKED.
 */
int qhk_execute_tact_cycle(qhk_core_t* core);

/**
 * Soft-reset operational state (does not clear forensic ledger on host side).
 */
int qhk_reset_kernel(qhk_core_t* core);

/**
 * Query current residual (mean abs gradient). Returns negative on error.
 */
double qhk_get_residual(const qhk_core_t* core);

#ifdef __cplusplus
}
#endif

#endif /* LIBDHQHK_H */
```

C SDK Implementation – libdhqhk.c

sdk/libdhqhk.c

```
/**
 * libdhqhk.c – Omni Quantum Dual Helix Core SDK (software reference model)
 * Version 1.1.0
 *
 * Implements the same semantics as the Python hardened kernel:
 *  - 4×12 helix torsion
 *  - baseline-protected GIP evaluation
```

```
 *  - SWAT fail-closed
 *  - residual + soft reset
 *
 * This is a pure-C host-side model suitable for simulation, HIL and
 * early bring-up of the Development Board. It does not talk to real silicon.
 */

#include "libdhqhk.h"

#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* Internal kernel state (not exposed in public header) */
typedef struct {
    double prev_torsion;
    int    baseline_set;
    double omega_crit;
    double lege_tol;
    unsigned long block_seq;
} qhk_internal_t;

static qhk_internal_t g_internal;

/* ------------------------------------------------------------------ */
/* Helpers                                                            */
/* ------------------------------------------------------------------ */

static void set_status(char *dst, const char *src)
{
    strncpy(dst, src, 31);
    dst[31] = '\0';
}

static double compute_residual(const qhk_core_t *core)
{
    if (!core || core->dimension < 2)
        return 0.0;

    double sum = 0.0;
    int n = core->dimension;
    /* central differences approximation of mean |gradient| */
    sum += fabs(core->state_psi[1] - core->state_psi[0]);
    for (int i = 1; i < n - 1; i++) {
        double g = 0.5 * (core->state_psi[i + 1] - core->state_psi[i - 1]);
        sum += fabs(g);
    }
    sum += fabs(core->state_psi[n - 1] - core->state_psi[n - 2]);
    return sum / (double)n;
}

static double calculate_torsion(const qhk_core_t *core)
{
    /* Map onto 4×12 lattice (pad / truncate to 48) */
    double lattice[48];
    memset(lattice, 0, sizeof(lattice));
    int n = core->dimension < 48 ? core->dimension : 48;
    for (int i = 0; i < n; i++)
        lattice[i] = core->state_psi[i];

    double torsion_sum = 0.0;
    for (int layer = 0; layer < 3; layer++) {
        for (int col = 0; col < 12; col++) {
            int idx_a = layer * 12 + col;
            int idx_b = (layer + 1) * 12 + ((col + 11) % 12); /* roll -1 */
            torsion_sum += fabs(lattice[idx_a] - lattice[idx_b]);
        }
    }
    return torsion_sum / 48.0;
}

/* Extremely lightweight JSON number array extractor for "state" or "psi".
 * Not a full parser – sufficient for the Intent-PDU format used by the project.
 */
static int extract_state_from_json(const char *json, double *out, int max_n)
{
    if (!json || !out)
        return -1;

    const char *p = strstr(json, "\"state\"");
    if (!p)
        p = strstr(json, "\"psi\"");
    if (!p)
        p = strstr(json, "\"initial_state\"");
    if (!p)
        return 0; /* no state field – not an error */
```

```
    p = strchr(p, '[');
    if (!p)
        return -1;
    p++;

    int count = 0;
    while (*p && count < max_n) {
        while (*p && (isspace((unsigned char)*p) || *p == ','))
            p++;
        if (*p == ']')
            break;
        char *end;
        double v = strtod(p, &end);
        if (end == p)
            break;
        out[count++] = v;
        p = end;
    }
    return count;
}

static int extract_dimension_from_json(const char *json)
{
    if (!json)
        return -1;
    const char *p = strstr(json, "\"dimension\"");
    if (!p)
        return -1;
    p = strchr(p, ':');
    if (!p)
        return -1;
    p++;
    while (*p && isspace((unsigned char)*p))
        p++;
    return (int)strtol(p, NULL, 10);
}

/* ------------------------------------------------------------------ */
/* Public API                                                         */
/* ------------------------------------------------------------------ */

int qhk_initialize_kernel(qhk_core_t *core)
{
    if (!core)
        return QHK_ERR_NULL;

    memset(core, 0, sizeof(*core));
    core->dimension = OQ_DHC_DIMENSION;
    set_status(core->swat_status, "CONNECTED");
    set_status(core->system_status, "VERIFIED_STABLE");
    set_status(core->wallet_lock, "SECURE");
    core->cycle_count = 0;

    memset(&g_internal, 0, sizeof(g_internal));
    g_internal.omega_crit = 2.5;
    g_internal.lege_tol   = 1.0e-6;
    g_internal.baseline_set = 0;
    g_internal.prev_torsion = 0.0;
    g_internal.block_seq = 0;

    return QHK_OK;
}

int qhk_reset_kernel(qhk_core_t *core)
{
    if (!core)
        return QHK_ERR_NULL;

    /* Preserve dimension, zero state, restore operational flags.
     * Forensic ledger (if kept on host) is intentionally not cleared. */
    int dim = core->dimension;
    memset(core->state_psi, 0, sizeof(core->state_psi));
    core->dimension = dim > 0 ? dim : OQ_DHC_DIMENSION;
    set_status(core->swat_status, "CONNECTED");
    set_status(core->system_status, "VERIFIED_STABLE");
    set_status(core->wallet_lock, "SECURE");
    core->cycle_count = 0;

    g_internal.prev_torsion = 0.0;
    g_internal.baseline_set = 0;
    /* block_seq intentionally continues */

    return QHK_OK;
}

int qhk_inject_intent_pdu(qhk_core_t *core, const char *json_pdu)
{
    if (!core)
        return QHK_ERR_NULL;
```

```
    if (!json_pdu)
        return QHK_ERR_INVALID_PDU;

    if (strcmp(core->swat_status, "PHYSICALLY_DISCONNECTED") == 0)
        return QHK_ERR_LOCKED;

    int dim = extract_dimension_from_json(json_pdu);
    if (dim >= 4 && dim <= OQ_DHC_DIMENSION)
        core->dimension = dim;

    double tmp[OQ_DHC_DIMENSION];
    int n = extract_state_from_json(json_pdu, tmp, OQ_DHC_DIMENSION);
    if (n < 0)
        return QHK_ERR_INVALID_PDU;

    if (n > 0) {
        memset(core->state_psi, 0, sizeof(core->state_psi));
        int copy = n < core->dimension ? n : core->dimension;
        for (int i = 0; i < copy; i++)
            core->state_psi[i] = tmp[i];
        /* New state → re-baseline on next cycle */
        g_internal.baseline_set = 0;
    }

    return QHK_OK;
}

int qhk_execute_tact_cycle(qhk_core_t *core)
{
    if (!core)
        return QHK_ERR_NULL;

    if (strcmp(core->swat_status, "PHYSICALLY_DISCONNECTED") == 0)
        return QHK_ERR_LOCKED;

    int n = core->dimension;
    if (n < 4 || n > OQ_DHC_DIMENSION)
        return QHK_ERR_DIMENSION;

    /* Discrete Laplace diffusion (matches Python 0.08 coefficient) */
    double lap[OQ_DHC_DIMENSION];
    for (int i = 0; i < n; i++) {
        double left  = core->state_psi[(i - 1 + n) % n];
        double right = core->state_psi[(i + 1) % n];
        lap[i] = left - 2.0 * core->state_psi[i] + right;
    }
    for (int i = 0; i < n; i++)
        core->state_psi[i] += 0.08 * lap[i];

    /* GIP / torsion evaluation */
    double current_torsion = calculate_torsion(core);
    double d_torsion_dt = 0.0;
    const double dt = 0.01;

    if (!g_internal.baseline_set) {
        g_internal.prev_torsion = current_torsion;
        g_internal.baseline_set = 1;
        d_torsion_dt = 0.0;
    } else {
        d_torsion_dt = (current_torsion - g_internal.prev_torsion) / dt;
        g_internal.prev_torsion = current_torsion;
    }

    double e_res = fabs(d_torsion_dt) - g_internal.omega_crit;
    double residual = compute_residual(core);

    if (e_res > 0.0) {
        /* Soft phase absorption */
        double phase = current_torsion * 3.141592653589793 * 0.15;
        for (int i = 0; i < n; i++) {
            /* real part of complex multiply by exp(i*phase) ≈ cos(phase) */
            core->state_psi[i] *= cos(phase);
        }
        residual = compute_residual(core);

        if (residual > g_internal.lege_tol * 50.0) {
            set_status(core->swat_status, "PHYSICALLY_DISCONNECTED");
            set_status(core->system_status, "LOCKDOWN_ABSORBING_STATE");
            set_status(core->wallet_lock, "ISOLATED_AND_LOCKED");
        }
    }

    g_internal.block_seq++;
    core->cycle_count++;
    return QHK_OK;
}

double qhk_get_residual(const qhk_core_t *core)
{
```

```
    if (!core)
        return -1.0;
    return compute_residual(core);
}
```

C SDK Test – test_sdk.c

sdk/test_sdk.c

```
/**
 * Minimal host-side test for libdhqhk (software model).
 * Build:  gcc -O2 -o test_sdk test_sdk.c libdhqhk.c -lm
 */
#include "libdhqhk.h"
#include <stdio.h>
#include <math.h>
#include <string.h>

static void print_status(const qhk_core_t *c)
{
    printf("  SWAT=%s  SYS=%s  WALLET=%s  residual=%.4e  cycle=%d\n",
           c->swat_status, c->system_status, c->wallet_lock,
           qhk_get_residual(c), c->cycle_count);
}

int main(void)
{
    qhk_core_t core;
    int rc;

    printf("=== OQ-DHC SDK C Test ===\n");

    rc = qhk_initialize_kernel(&core);
    if (rc != QHK_OK) {
        fprintf(stderr, "init failed: %d\n", rc);
        return 1;
    }
    printf("[1] After init:\n");
    print_status(&core);

    /* Load a smooth sine */
    for (int i = 0; i < OQ_DHC_DIMENSION; i++)
        core.state_psi[i] = sin(2.0 * 3.141592653589793 * i / 48.0);

    printf("[2] After sine load + 5 cycles:\n");
    for (int k = 0; k < 5; k++)
        qhk_execute_tact_cycle(&core);
    print_status(&core);

    if (strcmp(core.swat_status, "CONNECTED") != 0) {
        fprintf(stderr, "FAIL: expected CONNECTED after smooth evolution\n");
        return 1;
    }

    /* Massive injection */
    for (int i = 0; i < OQ_DHC_DIMENSION; i++)
        core.state_psi[i] += 200.0 * ((i % 3) - 1);
    printf("[3] After attack injection:\n");
    qhk_execute_tact_cycle(&core);
    qhk_execute_tact_cycle(&core); /* may need second cycle */
    print_status(&core);

    if (strcmp(core.swat_status, "PHYSICALLY_DISCONNECTED") != 0) {
        fprintf(stderr, "FAIL: expected PHYSICALLY_DISCONNECTED after attack\n");
        return 1;
    }

    /* Locked call must return error */
    rc = qhk_execute_tact_cycle(&core);
    if (rc != QHK_ERR_LOCKED) {
        fprintf(stderr, "FAIL: expected QHK_ERR_LOCKED\n");
        return 1;
    }
    printf("[4] Locked call correctly returned QHK_ERR_LOCKED\n");

    /* Reset */
    qhk_reset_kernel(&core);
    for (int i = 0; i < OQ_DHC_DIMENSION; i++)
        core.state_psi[i] = 0.3 * sin(2.0 * 3.141592653589793 * i / 48.0);
    qhk_execute_tact_cycle(&core);
    printf("[5] After reset + cycle:\n");
    print_status(&core);

    if (strcmp(core.swat_status, "CONNECTED") != 0) {
        fprintf(stderr, "FAIL: expected CONNECTED after reset\n");
        return 1;
```

```
    }

    printf("\n*** ALL C SDK TESTS PASSED ***\n");
    return 0;
}
```

C SDK Makefile

sdk/Makefile

```
# OQ-DHC SDK Makefile
CC      ?= gcc
CFLAGS  ?= -O2 -Wall -Wextra -std=c11
LDFLAGS ?= -lm

.PHONY: all test clean

all: test_sdk

test_sdk: test_sdk.c libdhqhk.c libdhqhk.h
	$(CC) $(CFLAGS) -o $@ test_sdk.c libdhqhk.c $(LDFLAGS)

test: test_sdk
	./test_sdk

clean:
	rm -f test_sdk *.o
```

KiCad Footprint Generator

kicad/scripts/gen_bga144_footprint.py

```
#!/usr/bin/env python3
"""
Generate a complete KiCad BGA-144 footprint (12×12, 0.8 mm pitch)
for the OQ-DHC package.

Usage:
  python gen_bga144_footprint.py [--out PATH] [--pitch 0.8] [--pad 0.4]

Default output:
  ../oq_dhc_devboard/footprints/BGA-144_12x12_0.8mm.kicad_mod
"""

from __future__ import annotations

import argparse
import os
from pathlib import Path

ROWS = "ABCDEFGHIJKL"  # 12 rows
N = 12


def generate(pitch: float = 0.8, pad_size: float = 0.4) -> str:
    half = (N - 1) / 2.0
    court = half * pitch + 0.8
    fab = half * pitch + 0.0

    lines: list[str] = [
        '(footprint "BGA-144_12x12_0.8mm"',
        '  (version 20221018)',
        '  (generator "gen_bga144_footprint.py")',
        '  (layer "F.Cu")',
        '  (descr "BGA-144 12x12 0.8mm pitch – full OQ-DHC package")',
        '  (tags "bga 144 oq-dhc 0.8mm")',
        '  (attr smd)',
        '  (fp_text reference "REF**" (at 0 -7.5) (layer "F.SilkS")',
        '    (effects (font (size 1 1) (thickness 0.15))))',
        '  (fp_text value "BGA-144_12x12_0.8mm" (at 0 7.5) (layer "F.Fab")',
        '    (effects (font (size 1 1) (thickness 0.15))))',
        f'  (fp_rect (start -{court:.3f} -{court:.3f}) (end {court:.3f} {court:.3f})',
        '    (stroke (width 0.1) (type default)) (fill none) (layer "F.CrtYd"))',
        f'  (fp_rect (start -{fab:.3f} -{fab:.3f}) (end {fab:.3f} {fab:.3f})',
        '    (stroke (width 0.1) (type default)) (fill none) (layer "F.Fab"))',
        # Pin-1 (A1) marker on silkscreen
        f'  (fp_circle (center {-half*pitch:.3f} {-half*pitch - 0.6:.3f}) '
        f'(end {-half*pitch:.3f} {-half*pitch - 0.35:.3f})',
        '    (stroke (width 0.1) (type default)) (fill none) (layer "F.SilkS"))',
    ]

    for ri, row in enumerate(ROWS):
        y = -half * pitch + ri * pitch
```

```
        for ci in range(1, N + 1):
            x = -half * pitch + (ci - 1) * pitch
            pad = f"{row}{ci}"
            lines.append(
                f'  (pad "{pad}" smd circle (at {x:.4f} {y:.4f}) '
                f'(size {pad_size} {pad_size}) (layers "F.Cu" "F.Paste" "F.Mask"))'
            )

    lines.append(")")
    return "\n".join(lines) + "\n"


def main() -> None:
    ap = argparse.ArgumentParser(description="Generate OQ-DHC BGA-144 KiCad footprint")
    ap.add_argument(
        "--out",
        type=Path,
        default=Path(__file__).resolve().parent.parent
        / "oq_dhc_devboard"
        / "footprints"
        / "BGA-144_12x12_0.8mm.kicad_mod",
    )
    ap.add_argument("--pitch", type=float, default=0.8)
    ap.add_argument("--pad", type=float, default=0.4)
    args = ap.parse_args()

    text = generate(pitch=args.pitch, pad_size=args.pad)
    args.out.parent.mkdir(parents=True, exist_ok=True)
    args.out.write_text(text, encoding="utf-8")
    pad_count = text.count('(pad "')
    print(f"Wrote {args.out}  ({pad_count} pads, pitch={args.pitch} mm)")


if __name__ == "__main__":
    main()
```

FPGA Bridge RTL – oq_dhc_fpga_bridge.sv

fpga/rtl/oq_dhc_fpga_bridge.sv

```
// ============================================================================
// OQ-DHC FPGA Emulation Bridge
// Maps the software-kernel semantics onto a synthesizable pin interface
// that mirrors the BGA-144 domains (Analog sense, Digital, SWAT, HS_IO, Clocks).
//
// Target: mid-range FPGA (e.g. Xilinx Artix-7 / Kintex, Lattice ECP5, etc.)
// This is a behavioural / soft model – not a tape-out netlist.
// ============================================================================

`timescale 1ns / 1ps

module oq_dhc_fpga_bridge #(
    parameter int DIMENSION   = 48,
    parameter int PHASES      = 12,
    parameter int DATA_W      = 32
) (
    // System
    input  logic                    clk_100m,       // 100 MHz system clock
    input  logic                    rst_n,

    // --- Analog domain (sense / monitor only on FPGA) ---
    output logic                    osc_carrier_p,  // A3
    output logic                    osc_carrier_n,  // A4
    output logic                    osc_mod_p,      // A5
    output logic                    osc_mod_n,      // A6

    // --- 12-phase hard tact (100 ms software / faster in sim) ---
    output logic [PHASES-1:0]       clk_phase,      // CLK_PHASE_01..12

    // --- SWAT interlock (to external TX-9942 driver) ---
    output logic                    swat_relais_out_p,  // F6
    output logic                    swat_relais_out_n,  // F7
    input  logic                    swat_sense,         // G6
    output logic                    swat_force_open,    // G7

    // --- High-speed I/O ring (simplified single-ended for bring-up) ---
    input  logic [12:0]             io_ring_in,
    output logic [12:0]             io_ring_out,
    output logic                    io_ring_oe,

    // --- Host / debug interface (AXI4-Lite style simplified) ---
    input  logic                    host_valid,
    input  logic [DATA_W-1:0]       host_wdata,
    input  logic [7:0]              host_addr,
    output logic                    host_ready,
    output logic [DATA_W-1:0]       host_rdata,
```

```
    // Status
    output logic [1:0]              swat_status_enc,    // 0=CONNECTED 1=DISCONNECTED
    output logic [1:0]              system_status_enc,
    output logic [1:0]              wallet_lock_enc
);

    // ---------------------------------------------------------------------
    // Internal state (mirrors qhk_core_t)
    // ---------------------------------------------------------------------
    typedef enum logic [1:0] {
        ST_CONNECTED              = 2'b00,
        ST_PHYSICALLY_DISCONNECTED = 2'b01
    } swat_e;

    typedef enum logic [1:0] {
        SYS_VERIFIED_STABLE       = 2'b00,
        SYS_LOCKDOWN              = 2'b01
    } sys_e;

    typedef enum logic [1:0] {
        WAL_SECURE                = 2'b00,
        WAL_ISOLATED              = 2'b01
    } wal_e;

    swat_e  swat_q,  swat_d;
    sys_e   sys_q,   sys_d;
    wal_e   wal_q,   wal_d;

    logic signed [15:0] state_psi [0:DIMENSION-1];
    logic [15:0]        residual_q;
    logic [31:0]        cycle_cnt;
    logic               baseline_set;
    logic signed [31:0] prev_torsion;
    logic [PHASES-1:0]  phase_cnt;

    // ---------------------------------------------------------------------
    // Clock phase generator (12 phases, sequential one-hot)
    // ---------------------------------------------------------------------
    always_ff @(posedge clk_100m or negedge rst_n) begin
        if (!rst_n) begin
            phase_cnt <= '0;
            clk_phase <= '0;
        end else begin
            phase_cnt <= phase_cnt + 1'b1;
            clk_phase <= 1'b1 << phase_cnt[3:0]; // simple rotating one-hot
        end
    end

    // ---------------------------------------------------------------------
    // Oscillator emulators (square-wave proxies for ω_c / ω_m)
    // ---------------------------------------------------------------------
    logic [15:0] carr_div, mod_div;
    always_ff @(posedge clk_100m or negedge rst_n) begin
        if (!rst_n) begin
            carr_div <= '0;
            mod_div  <= '0;
            osc_carrier_p <= 1'b0;
            osc_carrier_n <= 1'b1;
            osc_mod_p     <= 1'b0;
            osc_mod_n     <= 1'b1;
        end else begin
            carr_div <= carr_div + 1;
            mod_div  <= mod_div + 1;
            // Approximate dividers – not exact 98.112 Hz (would need DDS)
            if (carr_div == 16'd512) begin
                carr_div <= '0;
                osc_carrier_p <= ~osc_carrier_p;
                osc_carrier_n <= ~osc_carrier_n;
            end
            if (mod_div == 16'd12000) begin
                mod_div <= '0;
                osc_mod_p <= ~osc_mod_p;
                osc_mod_n <= ~osc_mod_n;
            end
        end
    end

    // ---------------------------------------------------------------------
    // Soft torsion / residual (combinational approximation)
    // ---------------------------------------------------------------------
    function automatic logic signed [31:0] calc_torsion();
        logic signed [31:0] acc;
        int layer, col, idx_a, idx_b;
        acc = 0;
        for (layer = 0; layer < 3; layer++) begin
            for (col = 0; col < 12; col++) begin
                idx_a = layer * 12 + col;
                idx_b = (layer + 1) * 12 + ((col + 11) % 12);
                if (idx_a < DIMENSION && idx_b < DIMENSION)
```

```
                acc += (state_psi[idx_a] > state_psi[idx_b]) ?
                       (state_psi[idx_a] - state_psi[idx_b]) :
                       (state_psi[idx_b] - state_psi[idx_a]);
        end
    end
    return acc / 48;
endfunction

// ------------------------------------------------------------------------
// Host register map (very small)
// 0x00 : control (bit0 = execute_cycle, bit1 = reset_kernel)
// 0x04 : status  (swat, sys, wallet packed)
// 0x08 : residual
// 0x0C : cycle_count
// 0x10+: state_psi write window (auto-increment optional)
// ------------------------------------------------------------------------
logic        exec_pulse, reset_pulse;
logic [7:0]  psi_wr_idx;

always_ff @(posedge clk_100m or negedge rst_n) begin
    if (!rst_n) begin
        host_ready   <= 1'b0;
        host_rdata   <= '0;
        exec_pulse   <= 1'b0;
        reset_pulse  <= 1'b0;
        psi_wr_idx   <= '0;
        for (int i = 0; i < DIMENSION; i++)
            state_psi[i] <= '0;
        swat_q       <= ST_CONNECTED;
        sys_q        <= SYS_VERIFIED_STABLE;
        wal_q        <= WAL_SECURE;
        residual_q   <= '0;
        cycle_cnt    <= '0;
        baseline_set <= 1'b0;
        prev_torsion <= '0;
    end else begin
        host_ready  <= 1'b0;
        exec_pulse  <= 1'b0;
        reset_pulse <= 1'b0;

        if (host_valid) begin
            host_ready <= 1'b1;
            case (host_addr)
                8'h00: begin
                    if (host_wdata[0]) exec_pulse  <= 1'b1;
                    if (host_wdata[1]) reset_pulse <= 1'b1;
                end
                8'h04: host_rdata <= {26'b0, wal_q, sys_q, swat_q};
                8'h08: host_rdata <= {16'b0, residual_q};
                8'h0C: host_rdata <= cycle_cnt;
                default: begin
                    // 0x10 and above: write state_psi
                    if (host_addr >= 8'h10 && host_addr < 8'h10 + DIMENSION) begin
                        state_psi[host_addr - 8'h10] <= host_wdata[15:0];
                        baseline_set <= 1'b0; // re-baseline after injection
                    end
                end
            endcase
        end

        // Soft reset
        if (reset_pulse) begin
            swat_q       <= ST_CONNECTED;
            sys_q        <= SYS_VERIFIED_STABLE;
            wal_q        <= WAL_SECURE;
            residual_q   <= '0;
            cycle_cnt    <= '0;
            baseline_set <= 1'b0;
            prev_torsion <= '0;
            for (int i = 0; i < DIMENSION; i++)
                state_psi[i] <= '0;
        end

        // Execute one tact cycle (simplified)
        if (exec_pulse && swat_q == ST_CONNECTED) begin
            logic signed [31:0] torsion, d_t, e_res;
            logic [15:0] res_acc;
            torsion = calc_torsion();

            if (!baseline_set) begin
                prev_torsion <= torsion;
                baseline_set <= 1'b1;
                d_t = 0;
            end else begin
                d_t = torsion - prev_torsion; // dt absorbed in threshold scale
                prev_torsion <= torsion;
            end

            e_res = (d_t > 0 ? d_t : -d_t) - 32'sd80; // scaled omega_crit
```

```
                // residual approx
                res_acc = 0;
                for (int i = 1; i < DIMENSION; i++)
                    res_acc += (state_psi[i] > state_psi[i-1]) ?
                               (state_psi[i] - state_psi[i-1]) :
                               (state_psi[i-1] - state_psi[i]);
                residual_q <= res_acc / DIMENSION;

                if (e_res > 0 && residual_q > 16'd200) begin
                    swat_q <= ST_PHYSICALLY_DISCONNECTED;
                    sys_q  <= SYS_LOCKDOWN;
                    wal_q  <= WAL_ISOLATED;
                end

                cycle_cnt <= cycle_cnt + 1;
            end
        end
    end

    // ---------------------------------------------------------------------
    // SWAT physical outputs
    // ---------------------------------------------------------------------
    always_comb begin
        // Active drive only while CONNECTED; released (high-Z / low) on cut
        if (swat_q == ST_CONNECTED) begin
            swat_relais_out_p = 1'b1;
            swat_relais_out_n = 1'b0;
            swat_force_open   = 1'b0;
        end else begin
            swat_relais_out_p = 1'b0;
            swat_relais_out_n = 1'b0;
            swat_force_open   = 1'b1; // force mechanical open
        end
    end

    // I/O ring: loopback / pass-through when connected, muted when locked
    assign io_ring_oe  = (swat_q == ST_CONNECTED);
    assign io_ring_out = io_ring_oe ? io_ring_in : '0;

    assign swat_status_enc   = swat_q;
    assign system_status_enc = sys_q;
    assign wallet_lock_enc   = wal_q;

endmodule
```

FPGA Testbench – tb_oq_dhc_bridge.sv

fpga/sim/tb_oq_dhc_bridge.sv

```
`timescale 1ns / 1ps

module tb_oq_dhc_bridge;
    logic clk = 0;
    logic rst_n = 0;
    always #5 clk = ~clk;

    logic        host_valid;
    logic [31:0] host_wdata;
    logic [7:0]  host_addr;
    logic        host_ready;
    logic [31:0] host_rdata;
    logic [1:0]  swat_status_enc, system_status_enc, wallet_lock_enc;
    logic        swat_relais_out_p, swat_relais_out_n, swat_force_open;
    logic        swat_sense = 1'b1;
    logic [11:0] clk_phase;
    logic        osc_carrier_p, osc_carrier_n, osc_mod_p, osc_mod_n;
    logic [12:0] io_ring_in = '0, io_ring_out;
    logic        io_ring_oe;

    oq_dhc_fpga_bridge dut (
        .clk_100m(clk),
        .rst_n(rst_n),
        .osc_carrier_p(osc_carrier_p),
        .osc_carrier_n(osc_carrier_n),
        .osc_mod_p(osc_mod_p),
        .osc_mod_n(osc_mod_n),
        .clk_phase(clk_phase),
        .swat_relais_out_p(swat_relais_out_p),
        .swat_relais_out_n(swat_relais_out_n),
        .swat_sense(swat_sense),
        .swat_force_open(swat_force_open),
        .io_ring_in(io_ring_in),
        .io_ring_out(io_ring_out),
        .io_ring_oe(io_ring_oe),
        .host_valid(host_valid),
        .host_wdata(host_wdata),
```

```
        .host_addr(host_addr),
        .host_ready(host_ready),
        .host_rdata(host_rdata),
        .swat_status_enc(swat_status_enc),
        .system_status_enc(system_status_enc),
        .wallet_lock_enc(wallet_lock_enc)
    );

    task automatic host_write(input [7:0] addr, input [31:0] data);
        host_addr  = addr;
        host_wdata = data;
        host_valid = 1;
        @(posedge clk);
        while (!host_ready) @(posedge clk);
        host_valid = 0;
        @(posedge clk);
    endtask

    initial begin
        host_valid = 0;
        host_wdata = 0;
        host_addr  = 0;
        rst_n = 0;
        repeat (10) @(posedge clk);
        rst_n = 1;
        repeat (5) @(posedge clk);

        // Load mild sine-like values
        for (int i = 0; i < 48; i++)
            host_write(8'h10 + i, $sin(2.0 * 3.1415926535 * i / 48.0) * 1000);

        // Execute a few cycles – should stay CONNECTED
        repeat (5) host_write(8'h00, 32'h1);
        $display("After smooth cycles: SWAT=%0d SYS=%0d WAL=%0d", swat_status_enc, system_status_enc, wallet_lock_enc);

        // Inject large disturbance
        for (int i = 0; i < 48; i++)
            host_write(8'h10 + i, 32'sd20000);
        host_write(8'h00, 32'h1);
        host_write(8'h00, 32'h1);
        $display("After attack: SWAT=%0d SYS=%0d WAL=%0d force_open=%0d",
                 swat_status_enc, system_status_enc, wallet_lock_enc, swat_force_open);

        // Reset
        host_write(8'h00, 32'h2);
        $display("After reset: SWAT=%0d", swat_status_enc);

        $display("TB finished");
        $finish;
    end
endmodule
```

FPGA Constraints – oq_dhc_pins.xdc

`fpga/constraints/oq_dhc_pins.xdc`

```
# OQ-DHC FPGA Emulation Bridge – Example pin constraints (Artix-7 style)
# Adjust package pins to your actual FPGA module / carrier.

## System
set_property PACKAGE_PIN E3  [get_ports clk_100m]
set_property IOSTANDARD LVCMOS33 [get_ports clk_100m]
create_clock -period 10.000 -name clk_100m [get_ports clk_100m]

set_property PACKAGE_PIN C2  [get_ports rst_n]
set_property IOSTANDARD LVCMOS33 [get_ports rst_n]

## SWAT interlock
set_property PACKAGE_PIN D4  [get_ports swat_relais_out_p]
set_property PACKAGE_PIN D3  [get_ports swat_relais_out_n]
set_property PACKAGE_PIN F4  [get_ports swat_sense]
set_property PACKAGE_PIN F3  [get_ports swat_force_open]
set_property IOSTANDARD LVCMOS33 [get_ports {swat_relais_out_p swat_relais_out_n swat_sense swat_force_open}]

## Clock phases (first 4 shown – extend as needed)
set_property PACKAGE_PIN G1  [get_ports {clk_phase[0]}]
set_property PACKAGE_PIN G2  [get_ports {clk_phase[1]}]
set_property PACKAGE_PIN H1  [get_ports {clk_phase[2]}]
set_property PACKAGE_PIN H2  [get_ports {clk_phase[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {clk_phase[*]}]

## Status LEDs
set_property PACKAGE_PIN G6  [get_ports {swat_status_enc[0]}]
set_property PACKAGE_PIN G5  [get_ports {swat_status_enc[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {swat_status_enc[*]}]
```

Vivado create_project.tcl

fpga/vivado/create_project.tcl

```
# ==============================================================================
# OQ-DHC FPGA Emulation Bridge – Vivado Project Generator
# Usage (from Vivado TCL shell or batch):
#   vivado -mode batch -source create_project.tcl
# ==============================================================================

set proj_name  "oq_dhc_fpga_bridge"
set proj_dir   [file normalize [file join [file dirname [info script]] "build"]]
set rtl_dir    [file normalize [file join [file dirname [info script]] ".." "rtl"]]
set constr_dir [file normalize [file join [file dirname [info script]] ".." "constraints"]]
set sim_dir    [file normalize [file join [file dirname [info script]] ".." "sim"]]

# Adjust part to your board (example: Artix-7 on Digilent Nexys/Arty)
set part "xc7a35tcsg324-1"

file mkdir $proj_dir
create_project $proj_name $proj_dir -part $part -force

set_property target_language Verilog [current_project]
set_property default_lib work [current_project]

# RTL
add_files -norecurse [list \
    [file join $rtl_dir "oq_dhc_fpga_bridge.sv"] \
]
set_property file_type SystemVerilog [get_files *.sv]
set_property top oq_dhc_fpga_bridge [current_fileset]

# Constraints
add_files -fileset constrs_1 -norecurse [list \
    [file join $constr_dir "oq_dhc_pins.xdc"] \
]

# Simulation
add_files -fileset sim_1 -norecurse [list \
    [file join $sim_dir "tb_oq_dhc_bridge.sv"] \
]
set_property file_type SystemVerilog [get_files -of_objects [get_filesets sim_1] *.sv]
set_property top tb_oq_dhc_bridge [get_filesets sim_1]

update_compile_order -fileset sources_1
update_compile_order -fileset sim_1

puts "OQ-DHC Vivado project created at: $proj_dir"
puts "Top: oq_dhc_fpga_bridge | Sim top: tb_oq_dhc_bridge | Part: $part"
```

Vivado run_sim.tcl

fpga/vivado/run_sim.tcl

```
# Run behavioural simulation (batch)
set proj_dir [file normalize [file join [file dirname [info script]] "build"]]
open_project [file join $proj_dir "oq_dhc_fpga_bridge.xpr"]
launch_simulation
run 50 us
puts "Simulation finished"
```

Vivado build_bitstream.tcl

fpga/vivado/build_bitstream.tcl

```
# Synth + impl + bitstream (batch)
set proj_dir [file normalize [file join [file dirname [info script]] "build"]]
open_project [file join $proj_dir "oq_dhc_fpga_bridge.xpr"]
launch_runs synth_1 -jobs 4
wait_on_run synth_1
launch_runs impl_1 -to_step write_bitstream -jobs 4
wait_on_run impl_1
puts "Bitstream run complete"
```

LiteX Platform

fpga/litex/oq_dhc_platform.py

```
#!/usr/bin/env python3
"""
LiteX platform description for the OQ-DHC FPGA Emulation Bridge.

This is a generic / board-agnostic platform that exposes the SWAT,
clock-phase, oscillator-proxy and host-register interfaces. Bind it to a
real board by overriding _io or by using a vendor platform and connecting
the bridge as a LiteX module.
"""

from litex.build.generic_platform import *
from litex.build.xilinx import Xilinx7SeriesPlatform
from litex.build.openocd import OpenOCD


_io = [
    # System
    ("clk100", 0, Pins("E3"), IOStandard("LVCMOS33")),
    ("cpu_reset", 0, Pins("C2"), IOStandard("LVCMOS33")),

    # SWAT interlock (example pins – remap to your carrier)
    ("swat", 0,
        Subsignal("relais_p", Pins("D4")),
        Subsignal("relais_n", Pins("D3")),
        Subsignal("sense",    Pins("F4")),
        Subsignal("force",    Pins("F3")),
        IOStandard("LVCMOS33"),
    ),

    # Status LEDs (encode SWAT / system)
    ("user_led", 0, Pins("G6"), IOStandard("LVCMOS33")),
    ("user_led", 1, Pins("G5"), IOStandard("LVCMOS33")),
    ("user_led", 2, Pins("G4"), IOStandard("LVCMOS33")),
    ("user_led", 3, Pins("G3"), IOStandard("LVCMOS33")),

    # Serial for host/debug
    ("serial", 0,
        Subsignal("tx", Pins("D10")),
        Subsignal("rx", Pins("A9")),
        IOStandard("LVCMOS33"),
    ),
]


class Platform(Xilinx7SeriesPlatform):
    default_clk_name   = "clk100"
    default_clk_period = 1e9 / 100e6

    def __init__(self, toolchain="vivado"):
        Xilinx7SeriesPlatform.__init__(self, "xc7a35tcsg324-1", _io, toolchain=toolchain)

    def create_programmer(self):
        return OpenOCD("openocd_xc7_ft2232.cfg", "bscan_spi_xc7a35t.bit")
```

LiteX Bridge Module

`fpga/litex/oq_dhc_bridge_module.py`

```
#!/usr/bin/env python3
"""
LiteX wrapper module around the SystemVerilog OQ-DHC FPGA bridge.

Exposes a CSR bank so the LiteX CPU (or host via UARTWishboneBridge)
can drive execute/reset, inject state_psi words, and read SWAT status,
residual and cycle count.
"""

from migen import *
from litex.soc.interconnect.csr import *


class OQDHCBridge(Module, AutoCSR):
    def __init__(self, platform, pads=None):
        self.platform = platform

        # --- Control / status CSRs ---
        self._control = CSRStorage(fields=[
            CSRField("execute", size=1, offset=0, pulse=True,
                     description="Pulse: run one tact cycle"),
            CSRField("reset",   size=1, offset=1, pulse=True,
                     description="Pulse: soft-reset kernel state"),
        ])
        self._status = CSRStatus(fields=[
            CSRField("swat",   size=2, offset=0,
                     description="0=CONNECTED 1=PHYSICALLY_DISCONNECTED"),
            CSRField("system", size=2, offset=2,
```

```
            description="0=VERIFIED_STABLE 1=LOCKDOWN"),
    CSRField("wallet", size=2, offset=4,
            description="0=SECURE 1=ISOLATED_AND_LOCKED"),
])
self._residual = CSRStatus(16, description="Mean-abs residual proxy")
self._cycle    = CSRStatus(32, description="Executed tact cycles")

# --- state_psi write window ---
self._psi_addr = CSRStorage(8, description="Index 0..47 into state_psi")
self._psi_data = CSRStorage(16, description="Signed 16-bit value to write")
self._psi_we   = CSRStorage(fields=[
    CSRField("write", size=1, offset=0, pulse=True,
            description="Pulse: write psi_data to psi_addr"),
])

# Signals to/from SV core
host_valid  = Signal()
host_addr   = Signal(8)
host_wdata  = Signal(32)
host_ready  = Signal()
host_rdata  = Signal(32)
swat_enc    = Signal(2)
sys_enc     = Signal(2)
wal_enc     = Signal(2)

# Simple host transaction FSM:
# idle -> write_ctrl / write_psi / read_status -> wait_ready -> idle
fsm = FSM(reset_state="IDLE")
self.submodules += fsm

do_exec   = Signal()
do_reset  = Signal()
do_psi_wr = Signal()
do_status = Signal()

self.sync += [
    do_exec.eq(self._control.fields.execute),
    do_reset.eq(self._control.fields.reset),
    do_psi_wr.eq(self._psi_we.fields.write),
]

# Status sampling every cycle from last host_rdata when addr was 0x04 etc.
# For simplicity we also wire status fields combinatorially from core outputs.
self.comb += [
    self._status.fields.swat.eq(swat_enc),
    self._status.fields.system.eq(sys_enc),
    self._status.fields.wallet.eq(wal_enc),
]

# Residual / cycle updated when we issue a status read (addr 0x08 / 0x0C)
# Minimal path: after every execute we issue internal reads.
read_phase = Signal(2)

fsm.act("IDLE",
    If(do_exec | do_reset,
        NextState("WR_CTRL")
    ).Elif(do_psi_wr,
        NextState("WR_PSI")
    )
)
fsm.act("WR_CTRL",
    host_valid.eq(1),
    host_addr.eq(0x00),
    host_wdata.eq(Cat(do_exec, do_reset, Replicate(0, 30))),
    If(host_ready,
        NextState("RD_STATUS")
    )
)
fsm.act("WR_PSI",
    host_valid.eq(1),
    host_addr.eq(0x10 + self._psi_addr.storage),
    host_wdata.eq(Cat(self._psi_data.storage, Replicate(0, 16))),
    If(host_ready,
        NextState("IDLE")
    )
)
fsm.act("RD_STATUS",
    host_valid.eq(1),
    host_addr.eq(0x04),
    If(host_ready,
        NextState("RD_RESIDUAL")
    )
)
fsm.act("RD_RESIDUAL",
    host_valid.eq(1),
    host_addr.eq(0x08),
    If(host_ready,
        NextValue(self._residual.status, host_rdata[:16]),
        NextState("RD_CYCLE")
```

```
            )
        )
        fsm.act("RD_CYCLE",
            host_valid.eq(1),
            host_addr.eq(0x0C),
            If(host_ready,
                NextValue(self._cycle.status, host_rdata),
                NextState("IDLE")
            )
        )

        # Pad defaults
        relais_p = getattr(pads, "relais_p", Signal()) if pads else Signal()
        relais_n = getattr(pads, "relais_n", Signal()) if pads else Signal()
        sense    = getattr(pads, "sense", Signal(reset=1)) if pads else Signal(reset=1)
        force    = getattr(pads, "force", Signal()) if pads else Signal()

        self.specials += Instance(
            "oq_dhc_fpga_bridge",
            i_clk_100m          = ClockSignal(),
            i_rst_n             = ~ResetSignal(),
            o_swat_relais_out_p = relais_p,
            o_swat_relais_out_n = relais_n,
            i_swat_sense        = sense,
            o_swat_force_open   = force,
            i_host_valid        = host_valid,
            i_host_wdata        = host_wdata,
            i_host_addr         = host_addr,
            o_host_ready        = host_ready,
            o_host_rdata        = host_rdata,
            o_swat_status_enc   = swat_enc,
            o_system_status_enc = sys_enc,
            o_wallet_lock_enc   = wal_enc,
            # unused outputs tied off
            o_osc_carrier_p     = Signal(),
            o_osc_carrier_n     = Signal(),
            o_osc_mod_p         = Signal(),
            o_osc_mod_n         = Signal(),
            o_clk_phase         = Signal(12),
            i_io_ring_in        = Signal(13),
            o_io_ring_out       = Signal(13),
            o_io_ring_oe        = Signal(),
        )

        platform.add_source(str(
            __import__("pathlib").Path(__file__).resolve().parent.parent
            / "rtl" / "oq_dhc_fpga_bridge.sv"
        ))
```

LiteX Target

`fpga/litex/target_oq_dhc.py`

```
#!/usr/bin/env python3
"""
LiteX SoC target that includes the OQ-DHC FPGA Emulation Bridge.

Examples:
  python target_oq_dhc.py --build
  python target_oq_dhc.py --build --load
  python target_oq_dhc.py --uart-name=serial --build
"""

import argparse
import os
import sys

# Allow running from fpga/litex without install
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from migen import *
from litex.soc.cores.clock import *
from litex.soc.integration.soc_core import *
from litex.soc.integration.builder import *
from litex.soc.cores.led import LedChaser

from oq_dhc_platform import Platform
from oq_dhc_bridge_module import OQDHCBridge


class BaseSoC(SoCCore):
    def __init__(self, sys_clk_freq=int(100e6), **kwargs):
        platform = Platform()

        SoCCore.__init__(
            self,
```

```
            platform,
            sys_clk_freq,
            ident="OQ-DHC LiteX SoC with FPGA Emulation Bridge",
            **kwargs,
        )

        # Clock / reset
        self.submodules.crg = CRG(
            platform.request("clk100"),
            ~platform.request("cpu_reset"),
        )

        # Status LEDs
        self.submodules.leds = LedChaser(
            pads=platform.request_all("user_led"),
            sys_clk_freq=sys_clk_freq,
        )

        # OQ-DHC Bridge (SWAT pads if present)
        try:
            swat_pads = platform.request("swat")
        except Exception:
            swat_pads = None
        self.submodules.oq_dhc = OQDHCBridge(platform, pads=swat_pads)


def main():
    parser = argparse.ArgumentParser(description="OQ-DHC LiteX Target")
    parser.add_argument("--build", action="store_true", help="Build bitstream")
    parser.add_argument("--load", action="store_true", help="Load bitstream")
    builder_args(parser)
    soc_core_args(parser)
    args = parser.parse_args()

    soc = BaseSoC(**soc_core_argdict(args))
    builder = Builder(soc, **builder_argdict(args))

    if args.build:
        builder.build()

    if args.load:
        prog = soc.platform.create_programmer()
        prog.load_bitstream(builder.get_bitstream_filename(mode="sram"))


if __name__ == "__main__":
    main()
```